PCET'S
Pimpri Chinchwad College of Engineering

R Commands: Getting Started

Faculty of Mathematics

Department of First Year Engineering

Mathematical Operations and Calculations

Basics

- `>` is the prompt sign in R.
- The assignment operators are the left arrow with dash <- and equal sign =.

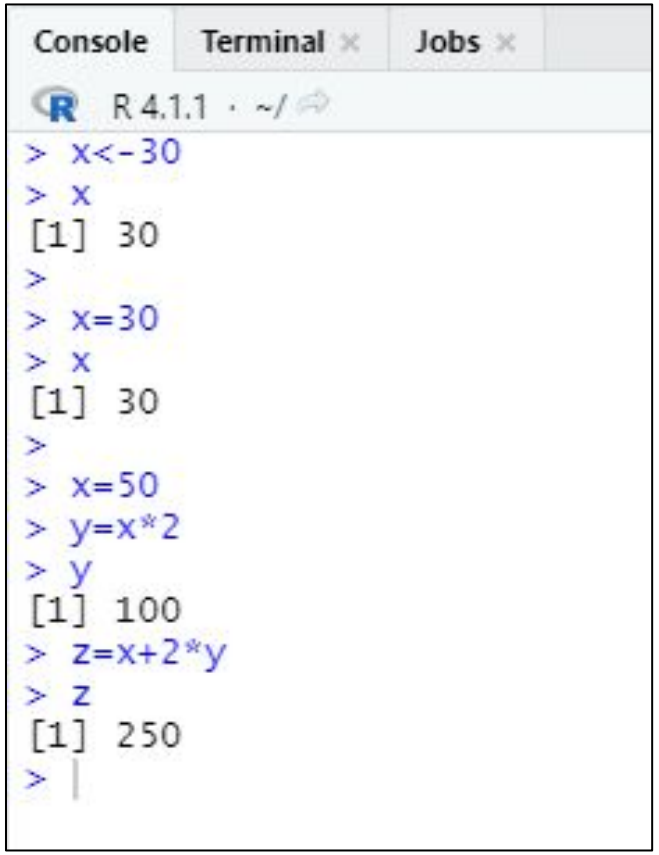
  `> x <- 30` assigns the value 30 to `x`.

  `> x = 30` assigns the value 30 to `x`.

- `> x = 50` assigns the value 50 to `x`.

  `> y = x * 2` assigns the value `2*x` to `y`.

  `> z = x + y` assigns the value `x + y` to `z`.

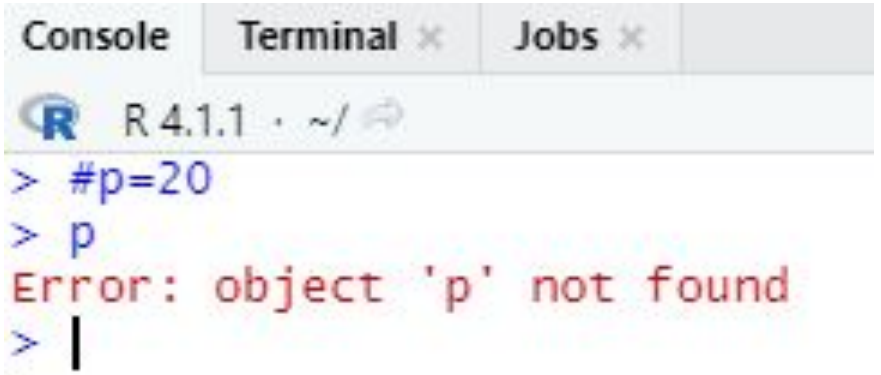


Basics

- **# : The character # marks the beginning of a comment. All characters until the end of the line are ignored.**

```
> # mu is the mean
```

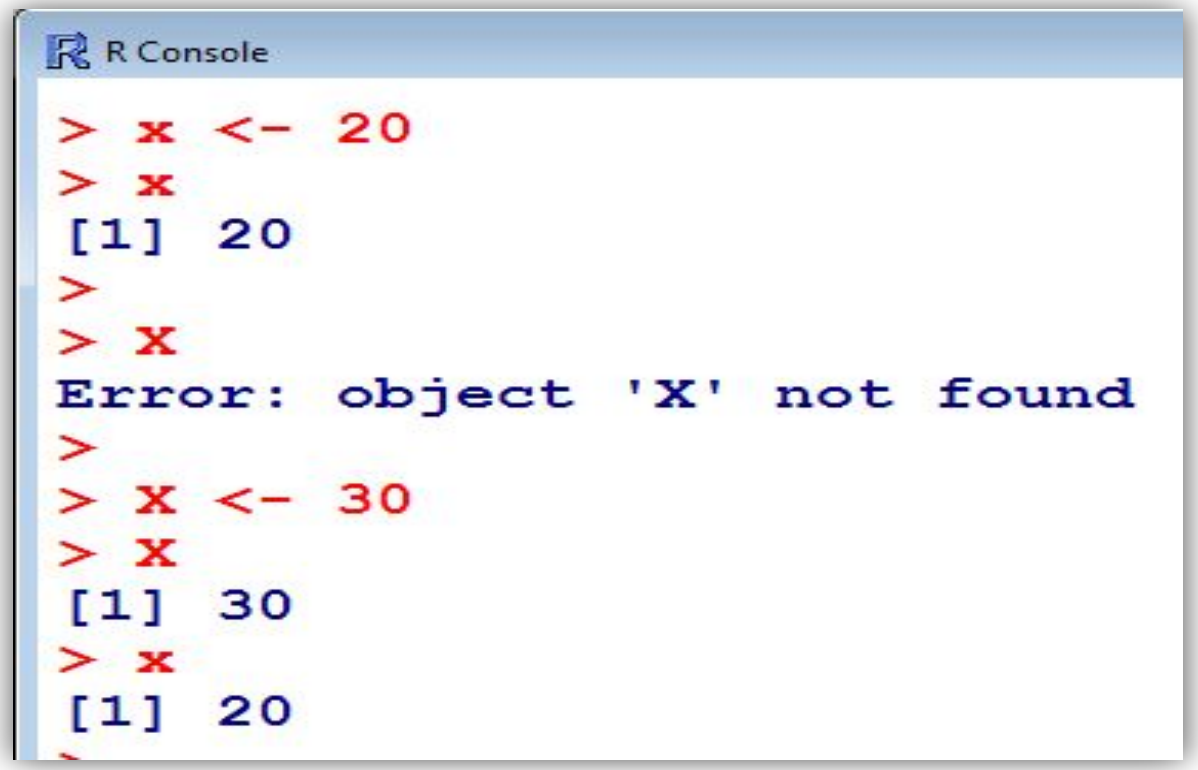
`> # x <- 20` is treated as comment only



Basics

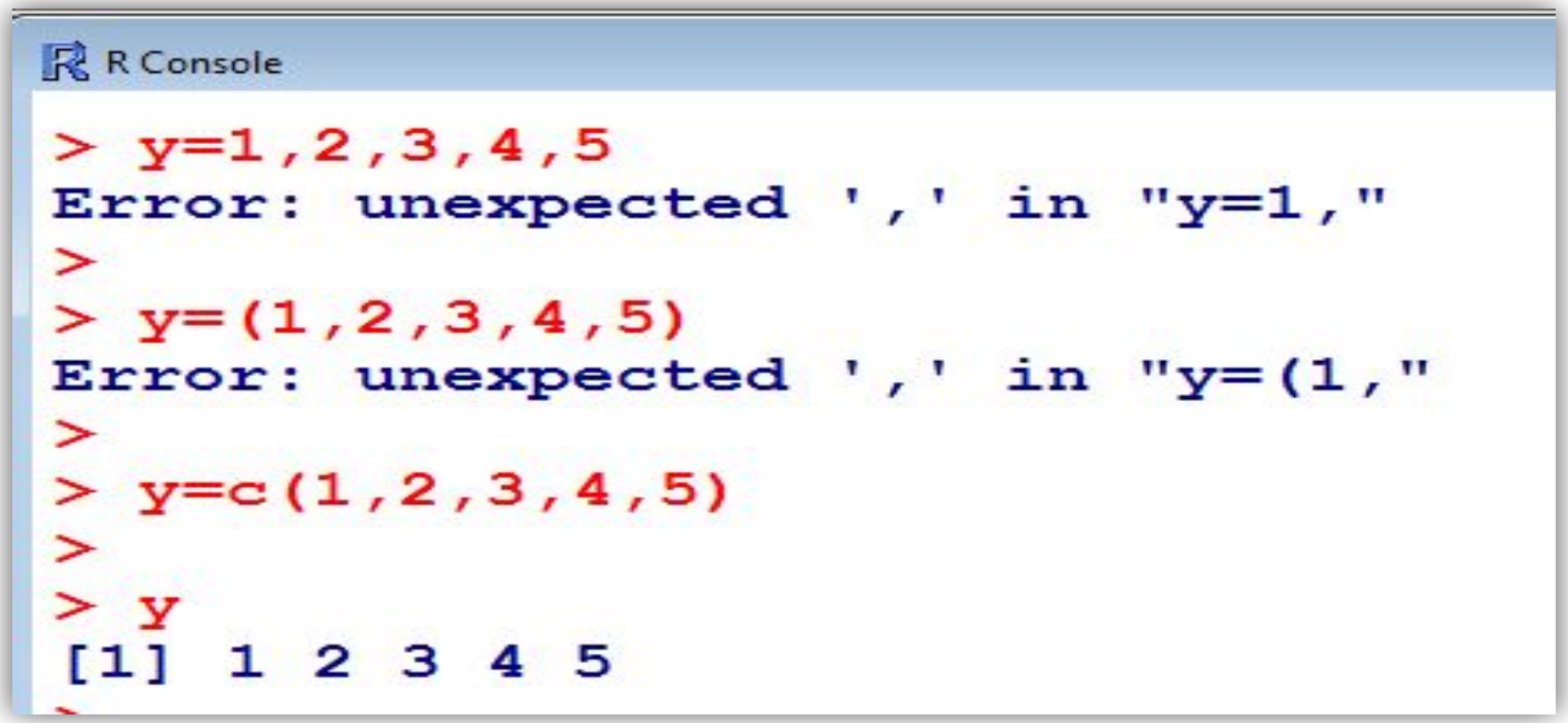
- Capital and small letters are different.

`> X <- 20` and `> x <- 20` are different



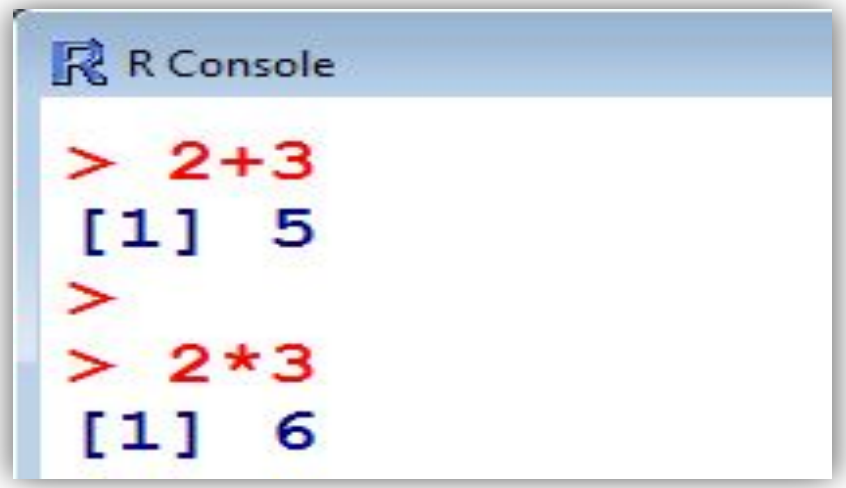
Basics

- **The command `c(1,2,3,4,5)` combines the numbers 1,2,3,4 and 5 to a vector.**



R as a calculator

```
> 2+3                    # Command
 [1] 5                   # Output
> 2*3                    # Command
 [1] 6                   # Output
```

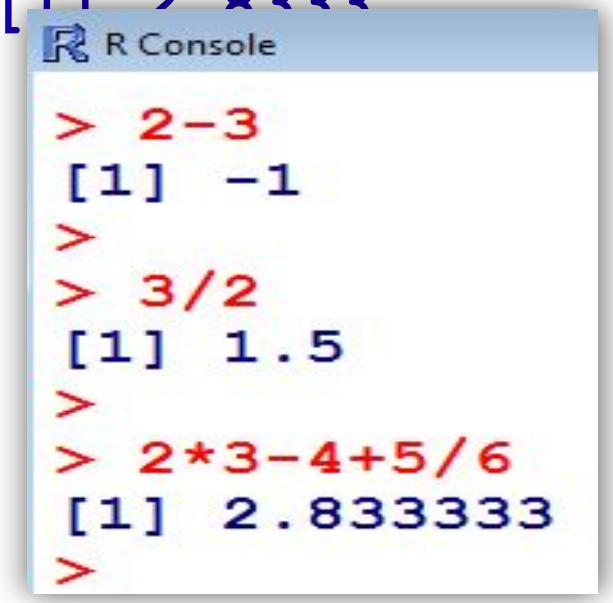


R as a calculator

```
> 2-3                 #  Command
 [1] -1               #  Output

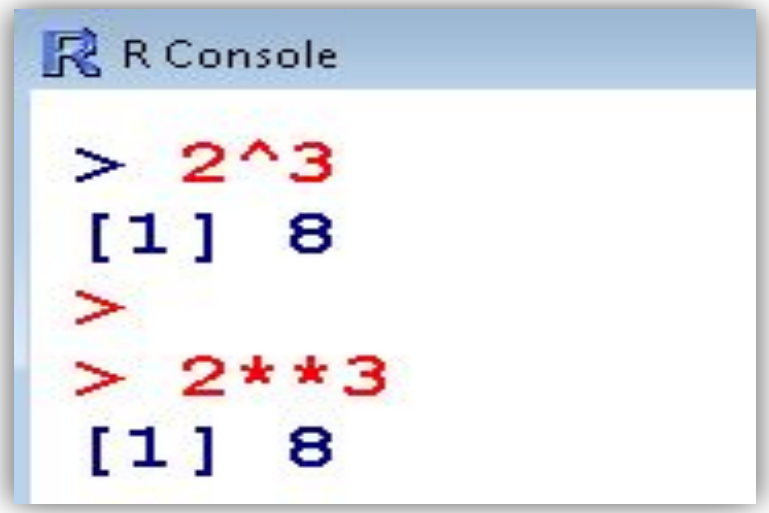
> 3/2                 #  Command
 [1] 1.5              #  Output

> 2*3-4+5/6           #  Command
 [1] 2.8333           #  Output
```



R as a calculator

```
> 2^3                    # Command
 [1] 8                   # Output
> 2**3                   # Command
 [1] 8                   # Output
```

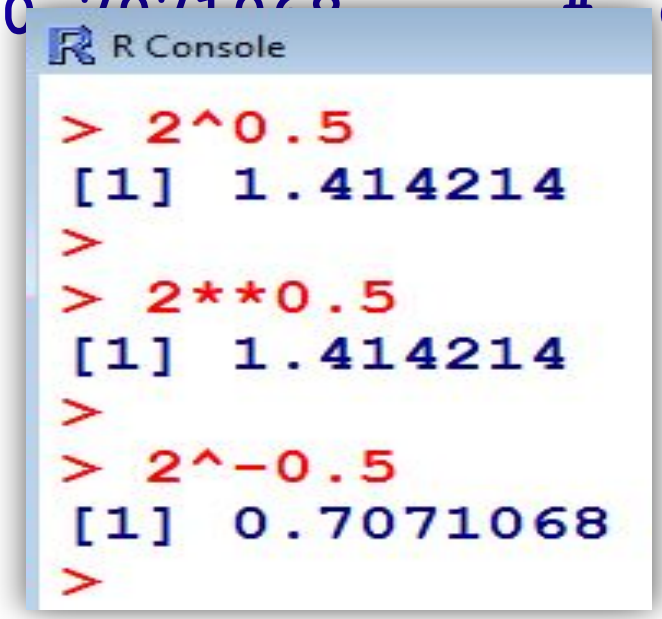


R as a calculator

```
> 2^0.5                 #  Command
[1] 1.414214            #  Output

> 2**0.5                #  Command
[1] 1.414214            #  Output

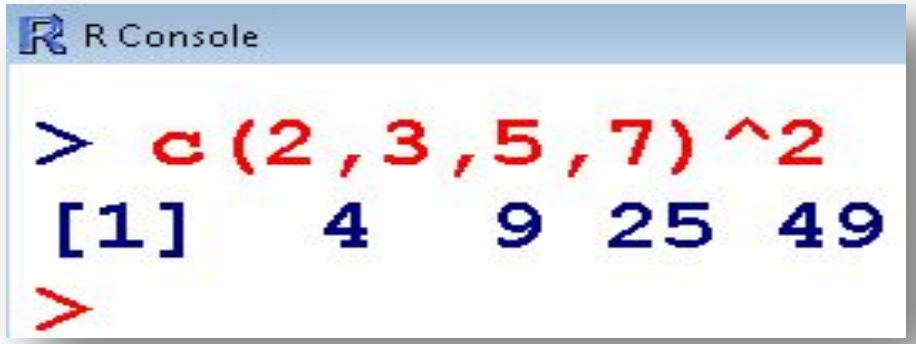
> 2^-0.5                #  Command
[1] 0.7071068           #  Output
```



R as a calculator

```
> c(2,3,5,7)^2                    # command: application to a
                                              vector
 [1]  4  9 25 49                            # output
```

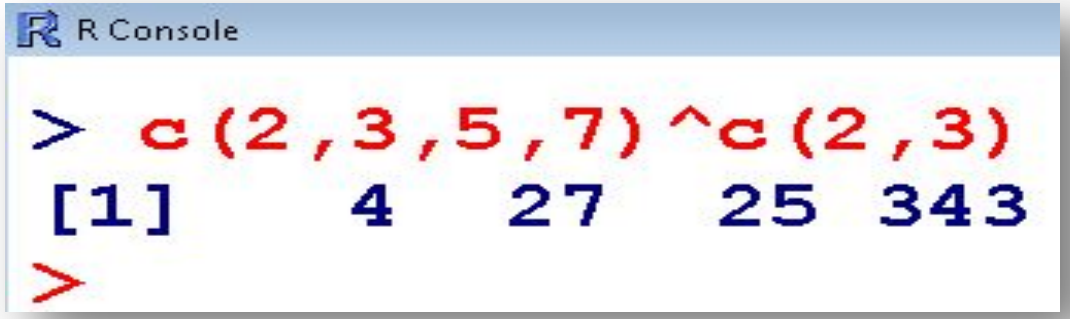
$2^2, \quad 3^2, 5^2, 7^2$



R as a calculator

```
> c(2,3,5,7)^c(2,3)          # !!ATTENTION! Observe the
                               operation
[1]   4 27 25 343            # output
```

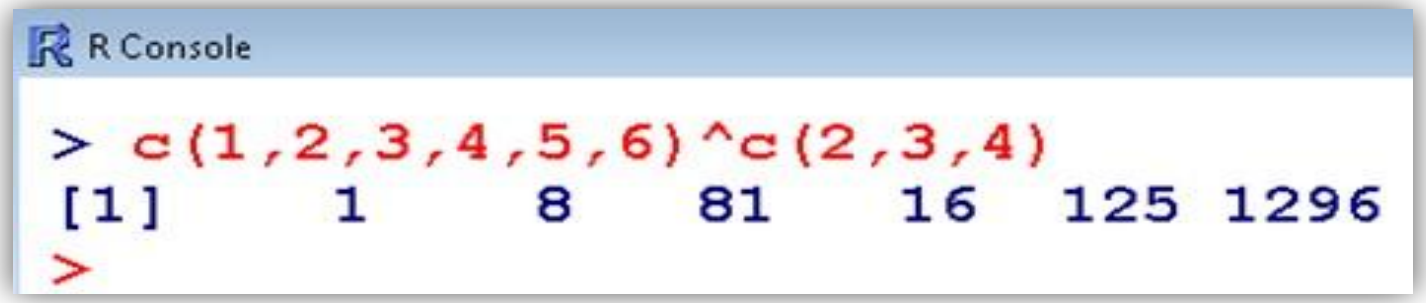
$2^2, \quad 3^3, 5^2, 7^3$



R as a calculator

```
> c(1,2,3,4,5,6)^c(2,3,4) # command: application
                          to a vector with vector
[1]  1 8 81 16 125 1296  # output
```

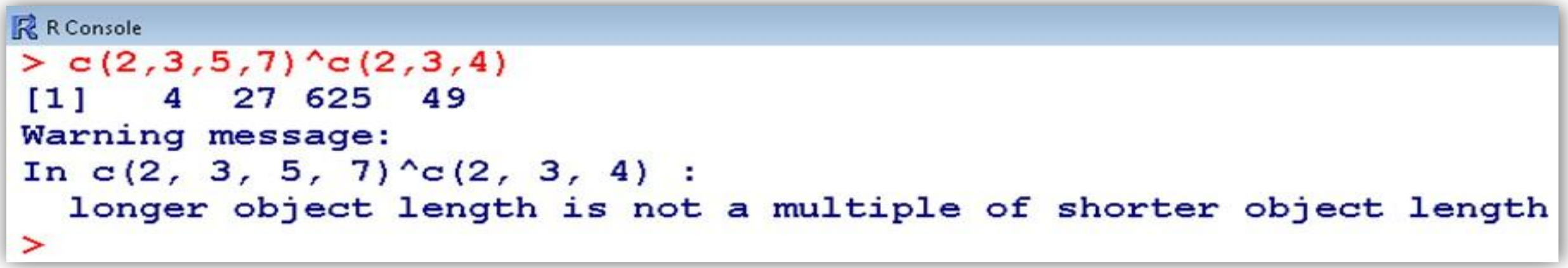
$1^2,\ \ 2^3,3^4,4^2,5^3,6^4$



R as a calculator

```
> c(2,3,5,7)^c(2,3,4)                    #error message   #
[1]  4 27 625 49                         output
Warning message:
longer object length is not a multiple of  shorter object
length in: c(2,3,5,7)^c(2,3,4)
```

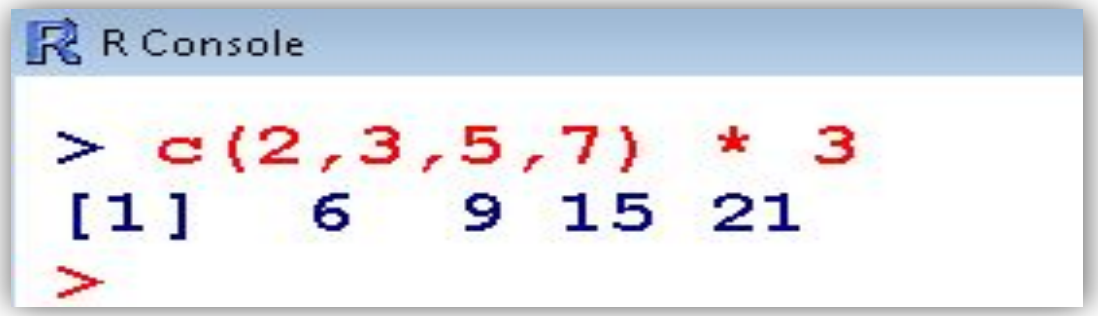
$2^2, \quad 3^3, 5^4, 7^2$



Multiplication and Division x * y, x/y

```
>c(2,3,5,7) * 3  [1]
 6  9 15 21
```

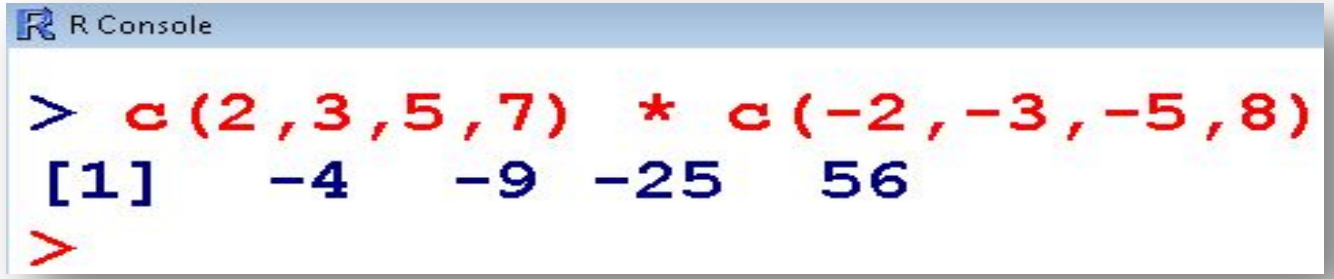
$2\times3,\ 3\times3,\ 5\times3,\ 7\times3$



Multiplication and Division x * y, x/y

```
>c(2,3,5,7) * c(-2,-3,-5,8)  [1] -4
-9 -25 56
```

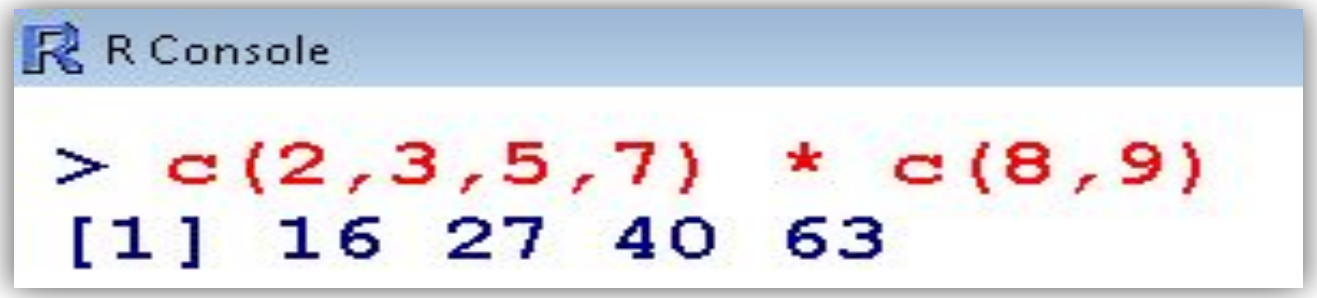
$$2\times(-2),\ 3\times(-3),\ 5\times(-5),\ 7\times 8$$



Multiplication and Division x * y, x/y

```
> c(2,3,5,7)      * c(8,9)     #  !!!   ATTENTION
[1] 16 27 40      63
```

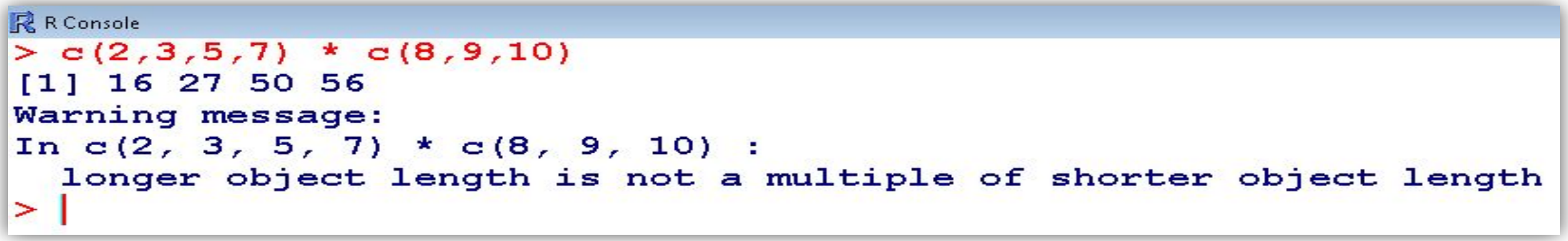
$2\times8,\ \ 3\times9,\ \ 5\times8,\ \ 7\times9$



Multiplication and Division x * y, x/y

```
>c(2,3,5,7) * c(8,9,10) # error message  [1] 16 27 50 56
    Warning message:
    longer object length
    is not a multiple of shorter object length  in: c(2, 3,
    5, 7) * c(8, 9, 10)
```

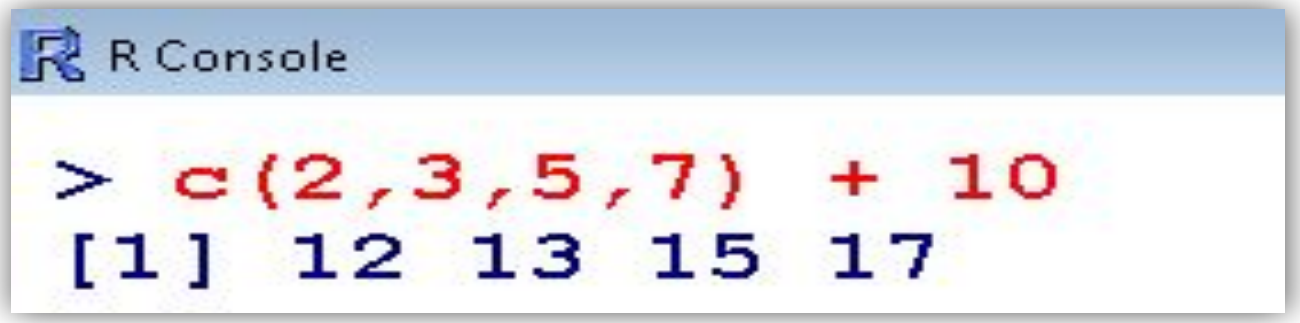
$2\times8,\ 3\times9,\ \ 5\times10, 7\times8$



Addition and Subtraction x + y, x − y

```
> c(2,3,5,7) + 10
[1] 12 13 15 17
```

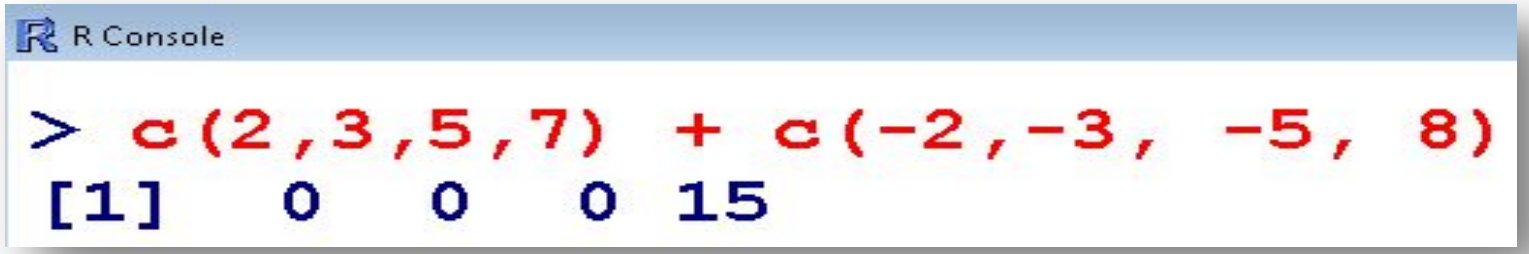
$2 + 10, \quad 3 + 10, \quad 5 + 10, \quad 7 + 10$



Addition and Subtraction x + y, x − y

```
> c(2,3,5,7) + c(-2,-3, -5, 8)
[1] 0 0 0 15
```

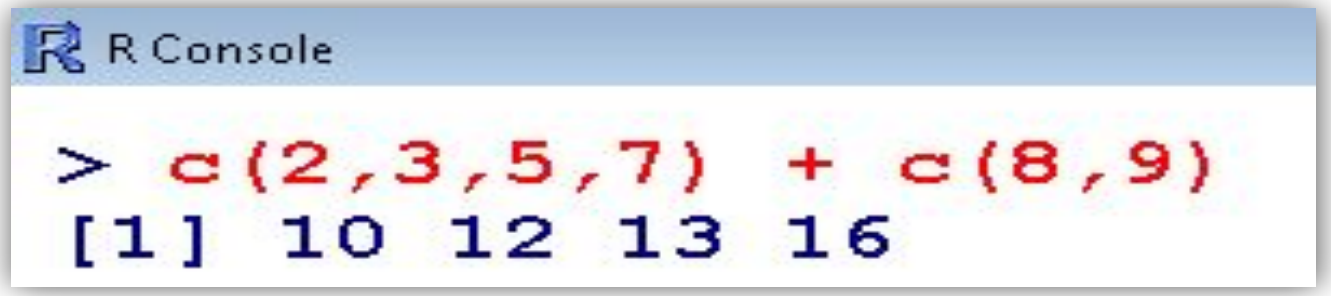
$$2+(-2),\quad 3+(-3),5+(-5),7+8$$



Addition and Subtraction x + y, x − y

```
> c(2,3,5,7)      +  c(8,9)    #  !!!  ATTENTION!
 [1] 10 12 13     1
                  6
```

$$2 + 8, 3 + 9, 5 + 2, 7 + 9$$



Addition and Subtraction x + y, x − y

```
>c(2,3,5,7) + c(8,9,10) # error message  [1] 10 12 15 15
  Warning message:
  longer object length
   is not a multiple of shorter object length  in: c(2, 3,
   5, 7) + c(8, 9, 10)
```

$$2+8,\quad 3+9,5+10,\quad 7+8$$

R Console

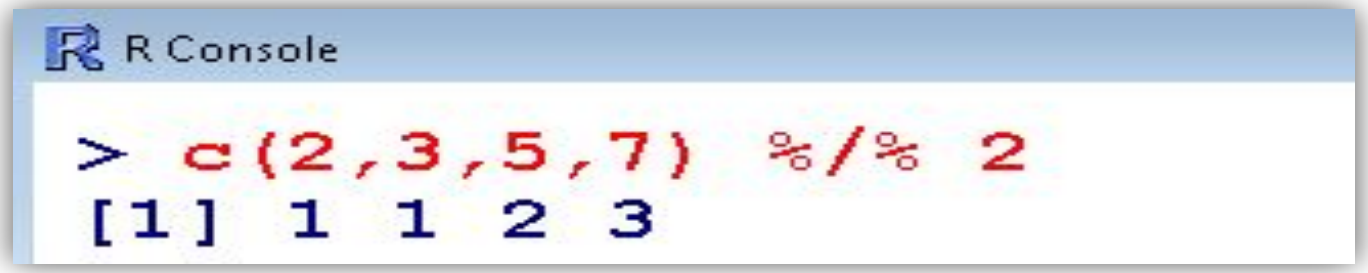
```
> c(2,3,5,7) + c(8,9,10)
[1] 10 12 15 15
Warning message:
In c(2, 3, 5, 7) + c(8, 9, 10) :
  longer object length is not a multiple of shorter object length
>
```

Integer Division %/%

Integer Division: Division in which the fractional part (remainder) is discarded

```
>c(2,3,5,7) %/% 2   [1] 1 1 2 3
```

2%/%2, 3%/%2, 5%/%2, 7%/%2



Integer Division %/%

Integer Division: Division in which the fractional part (remainder) is discarded

```
>c(2,3,5,7) %/% c(2,3)   [1] 1 1 2 2
```

2%/%2, 3%/%3, 5%/%2, 7%/%3

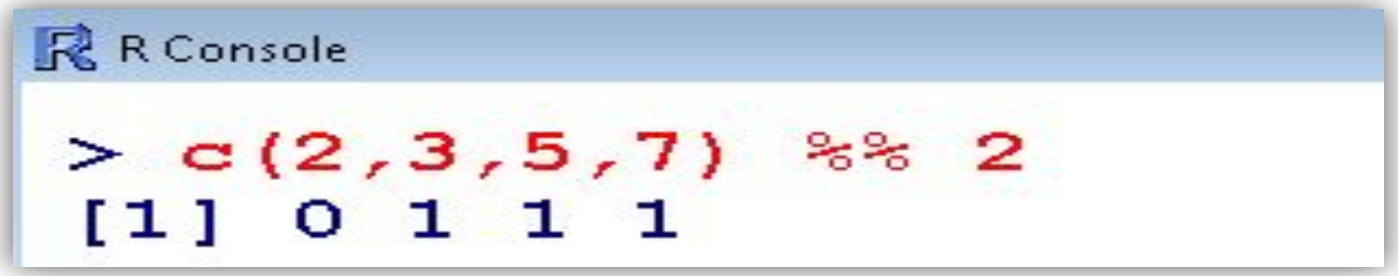


Modulo Division (x mod y) %%:

x mod y : modulo operation finds the remainder after division of one number by another

```
> c(2,3,5,7) %% 2
[1] 0 1 1 1
```

2%%2,3%%2, 5%%2, 7%%2

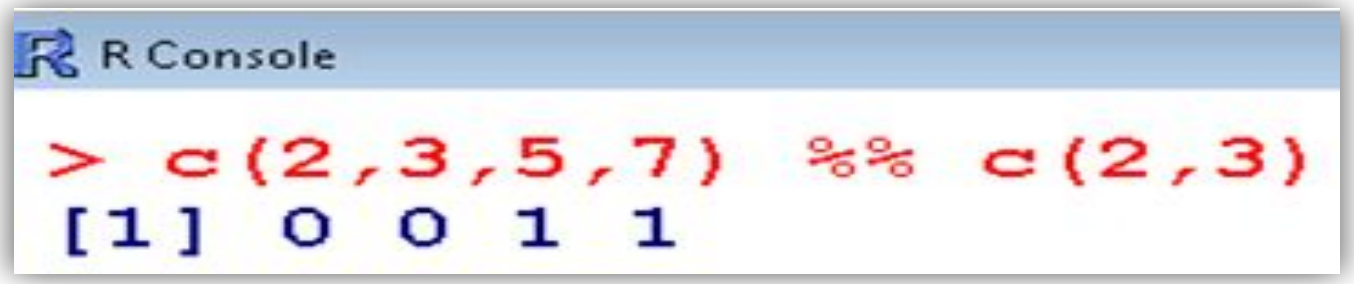


Modulo Division (x mod y) %%:

x mod y : modulo operation finds the remainder after division of one number by another

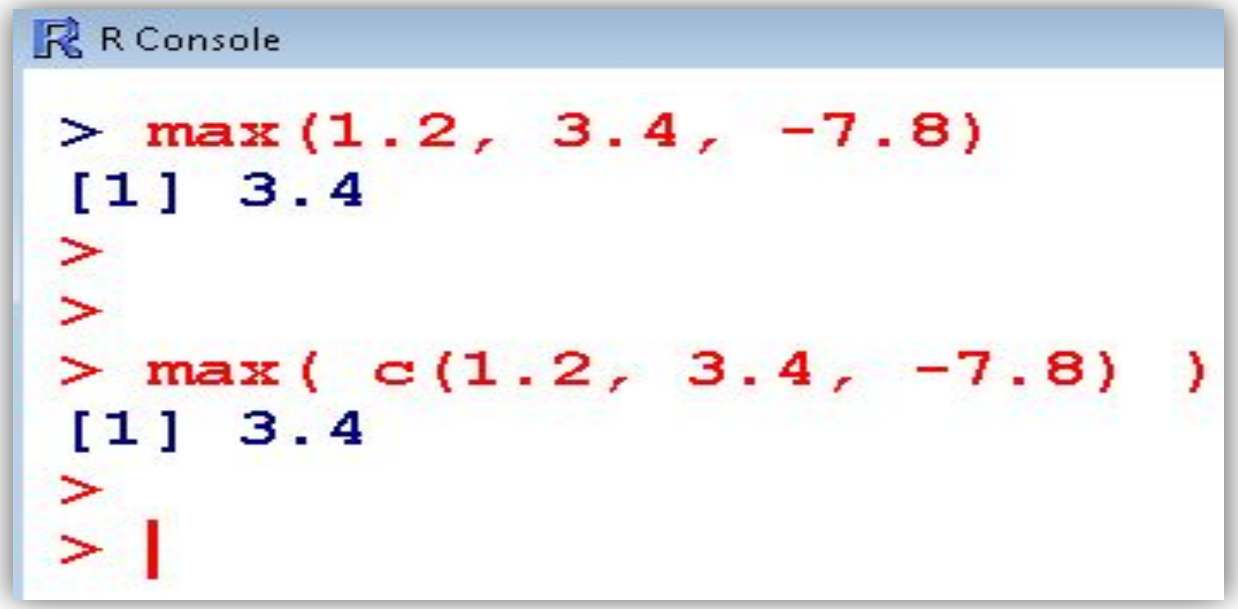
```
> c(2,3,5,7) %% c(2,3)  [1] 0 0 1 1
```

2%%2, 3%%3, 5%%2, 7%%3



Maximum: max

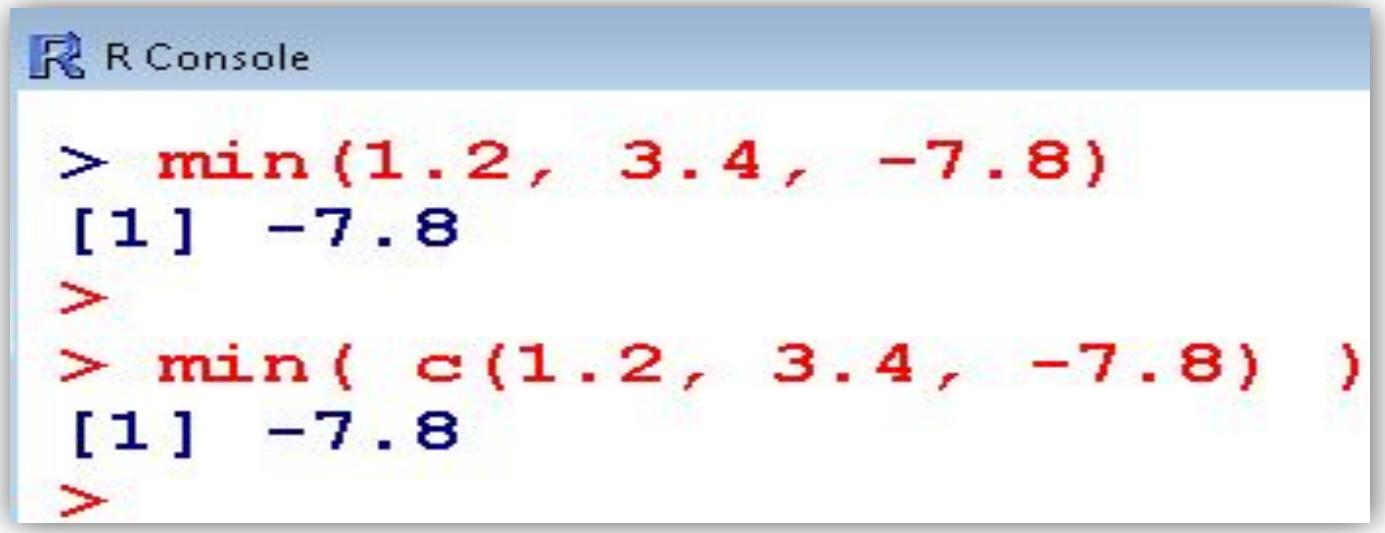
```
> max(1.2, 3.4, -7.8)
[1] 3.4
```




```
> max( c(1.2, 3.4, -7.8) )
[1] 3.4
```

Minimum : min

```
> min(1.2, 3.4, -7.8)
[1] -7.8
```




```
> min( c(1.2, 3.4, -7.8) )
[1] -7.8
```

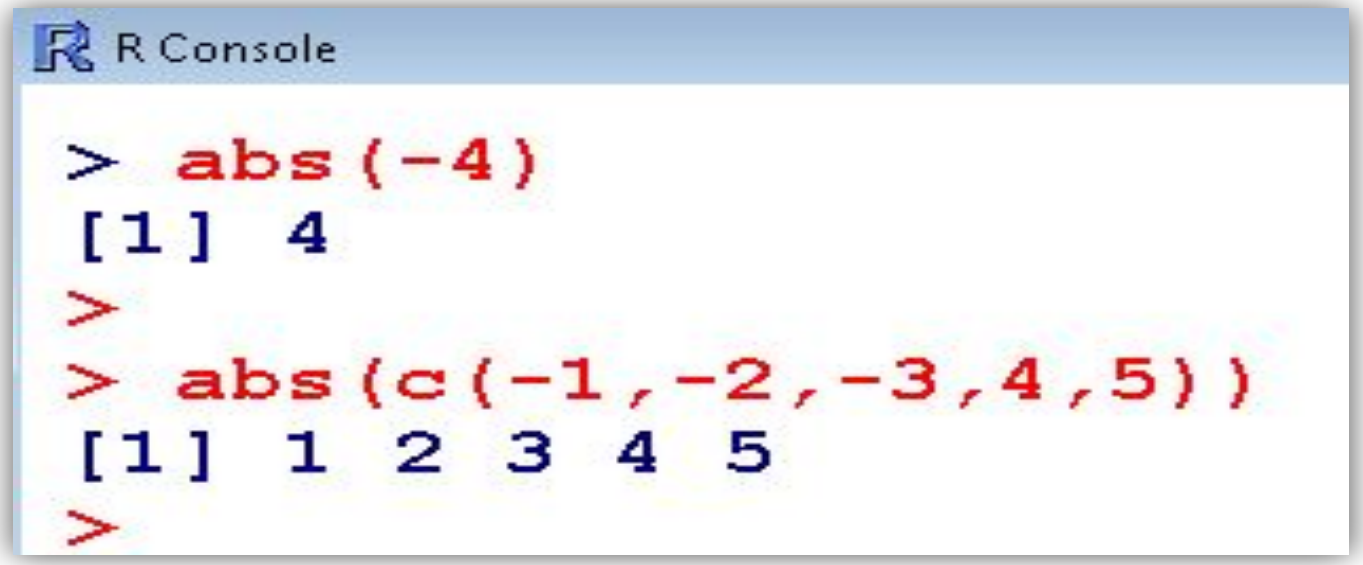
Overview Over Further Functions

`abs()`	Absolute value
`sqrt()`	Square root
`round(), floor(), ceiling()`	Rounding, up and down
`sum(), prod()`	Sum and product
`log(), log10(), log2()`	Logarithms
`exp()`	Exponential function
`sin(), cos(), tan(),` `asin(), acos(), atan()`	Trigonometric functions
`sinh(), cosh(), tanh(),` `asinh(), acosh(), atanh()`	Hyperbolic functions

Examples

```
> abs(-4)
[1] 4

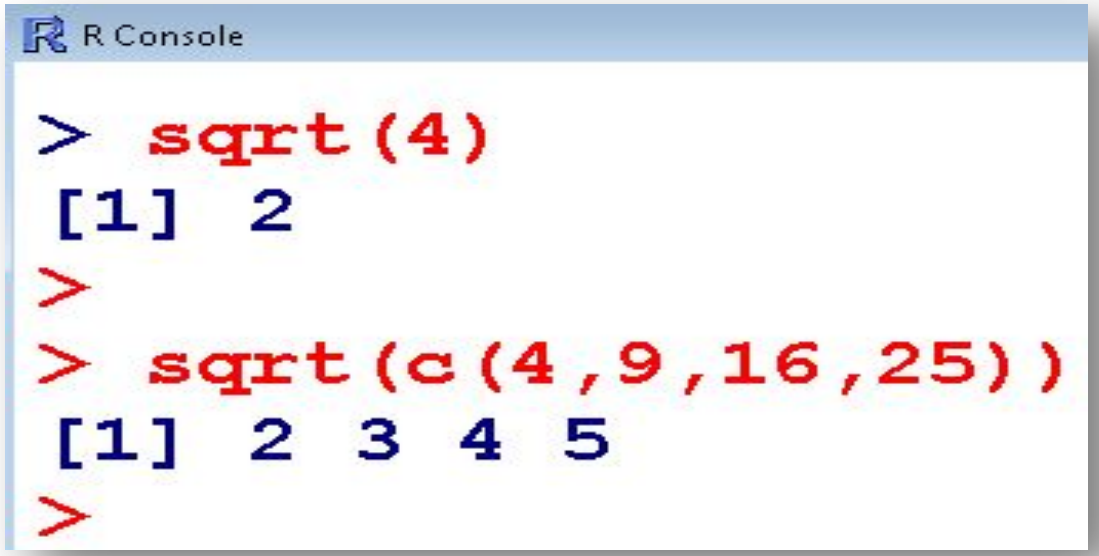
>abs(c(-1,-2,-3,4,5))   [1] 1
2 3 4 5
```



Examples

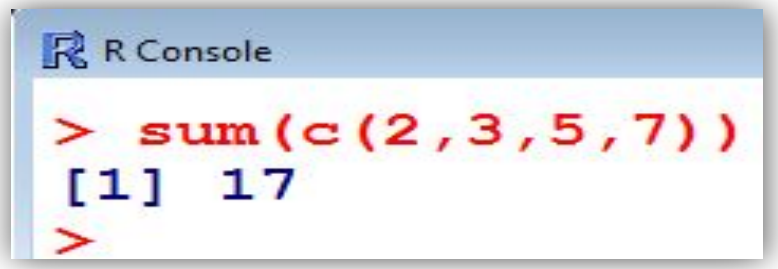
```
> sqrt(4)
[1] 2

>sqrt(c(4,9,16,25))  [1] 2
3 4 5
```

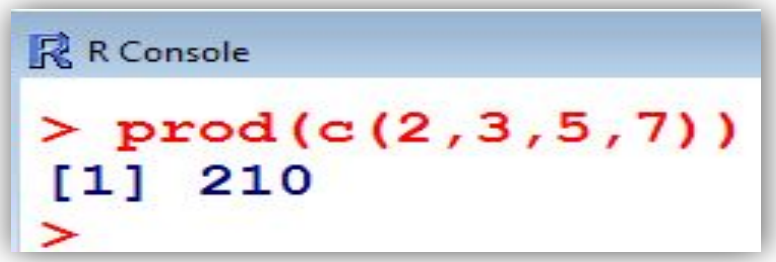


Examples

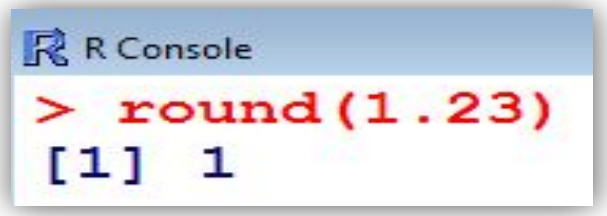
```
>sum(c(2,3,5,7))   [1] 17
```



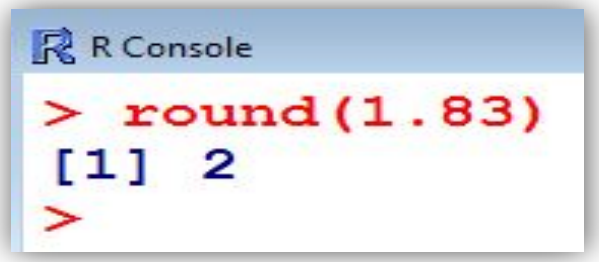

```
>prod(c(2,3,5,7))   [1]
210
```




```
> round(1.23)
[1] 1
```




```
> round(1.83)
[1] 2
```



Assignments

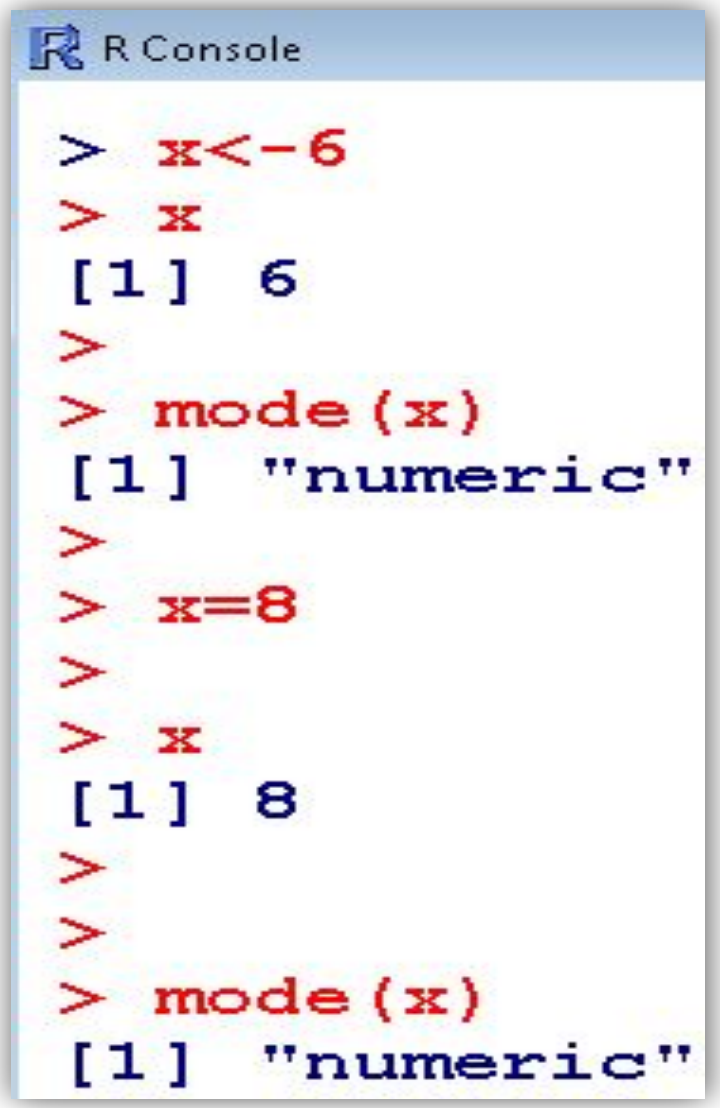
Assignments can be made in two ways:

```
> x<-6
> x
 [1] 6

> mode(x)
 1    "numeric"

> x=8
> x
 [1] 8

> mode(x)
 1    "numeric"
```



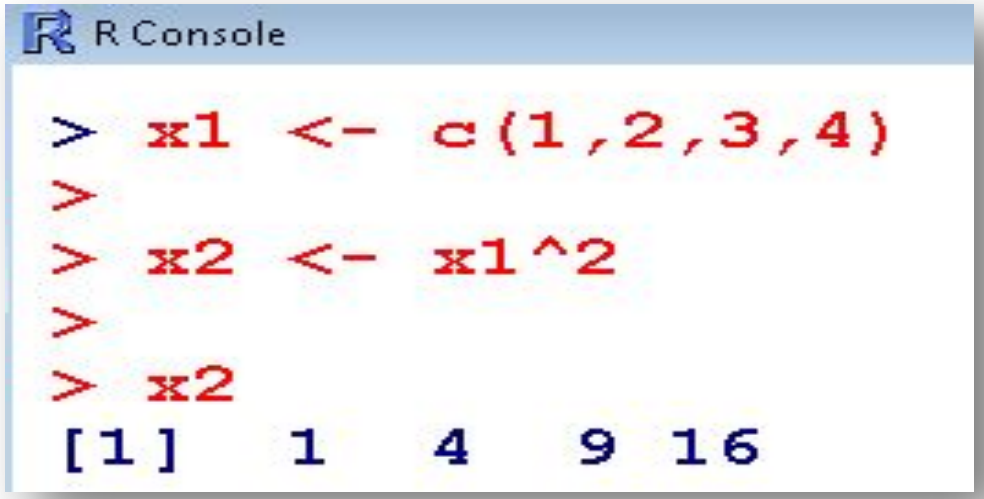
Assignments

An assignment can also be used to save values in variables:

```
> x1   <- c(1,2,3,4)

> x2   <- x1^2


> x2
 [1]  1  4  9 16
```



ATTENTION: R is case sensitive (X is not the same as x)

Functions

- **Functions are a bunch of commands grouped together in a sensible unit**
- **Functions take input arguments, do calculations (or make some graphics, call other functions) and produce some output and return a result in a variable. The returned variable can be a complex construct, like a list.**

Functions

Syntax

```
Name <- function(Argument1, Argument2, ...)
        {
         expression
                                  }
```

where expression is a single command or a group of commands

Function arguments with description and default values

- **Function arguments can be given a meaningful name**
- **Function arguments can be set to default values**
- **Functions can have the special argument '...'**

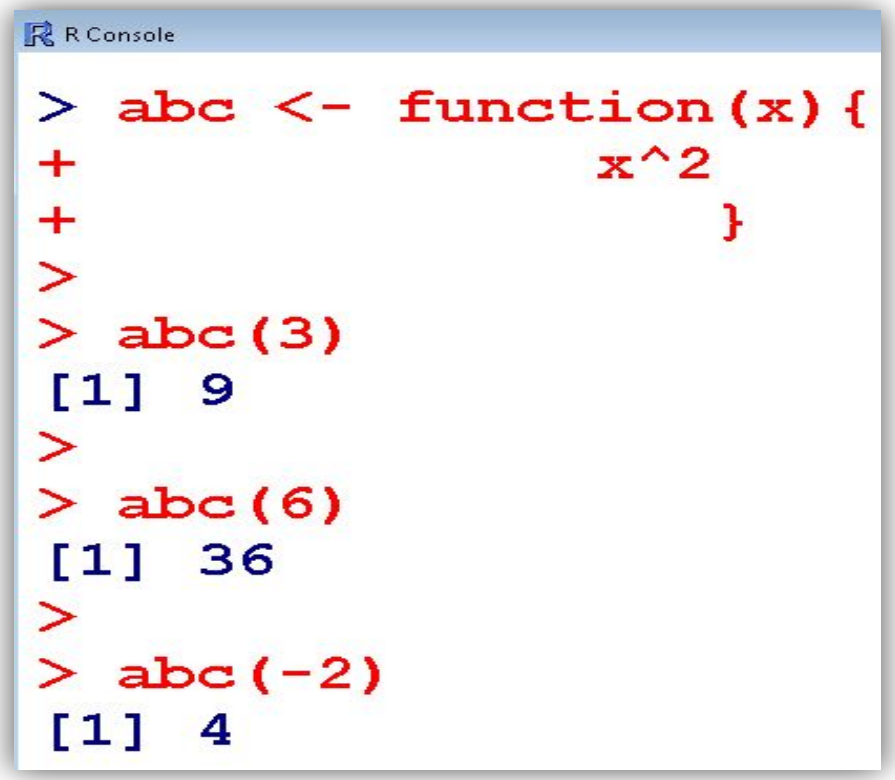
Functions (Single variable)

The sign <- is furthermore used for defining functions:

```
> abc <- function(x){
                    x^2
                     }
```

```
> abc(3)
[1] 9
```

```
> abc(-2)
[1] 4
```




```
> abc(6)
[1] 36
```

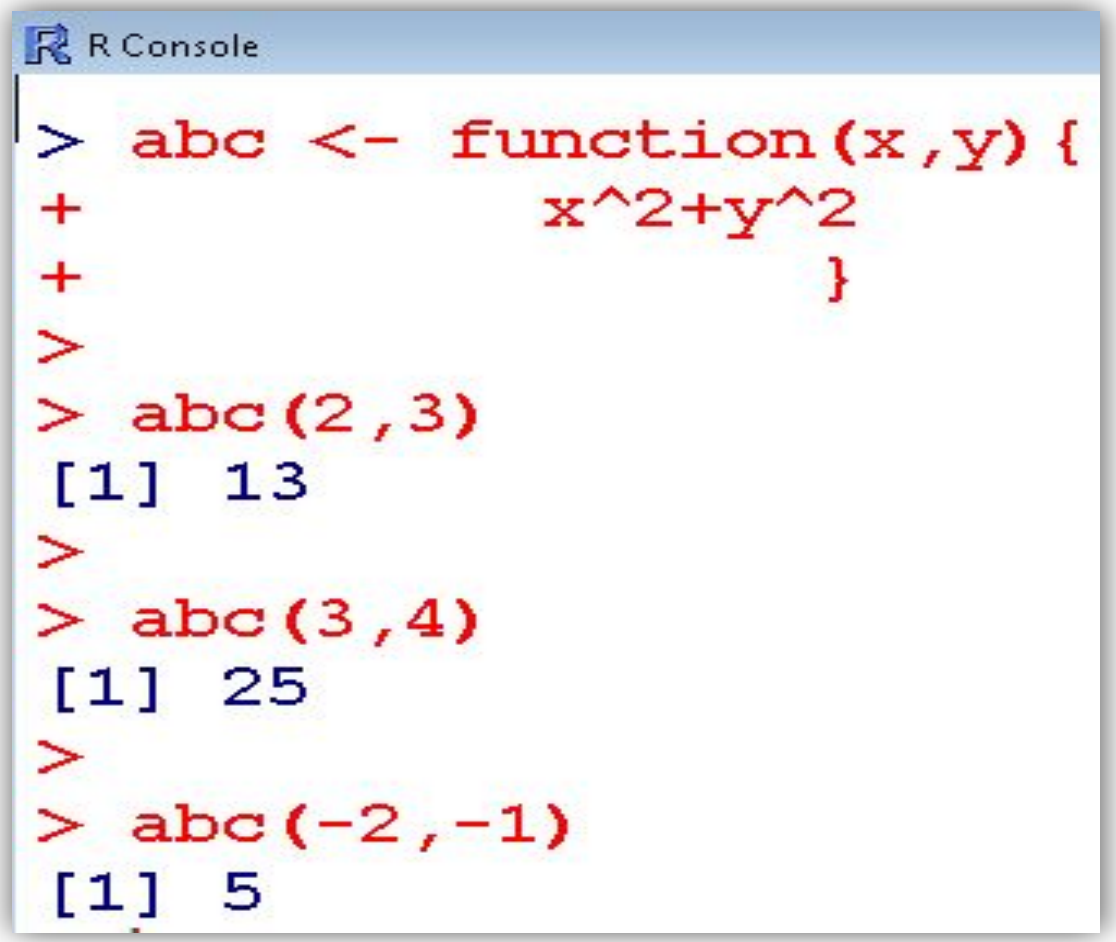
Functions (Two variables)

```
> abc <- function(x,y){
                        x^2+y^2
                              }

> abc(2,3)   [1] 13

> abc(3,4)   [1] 25

> abc(-2,-1)
  [1] 5
```



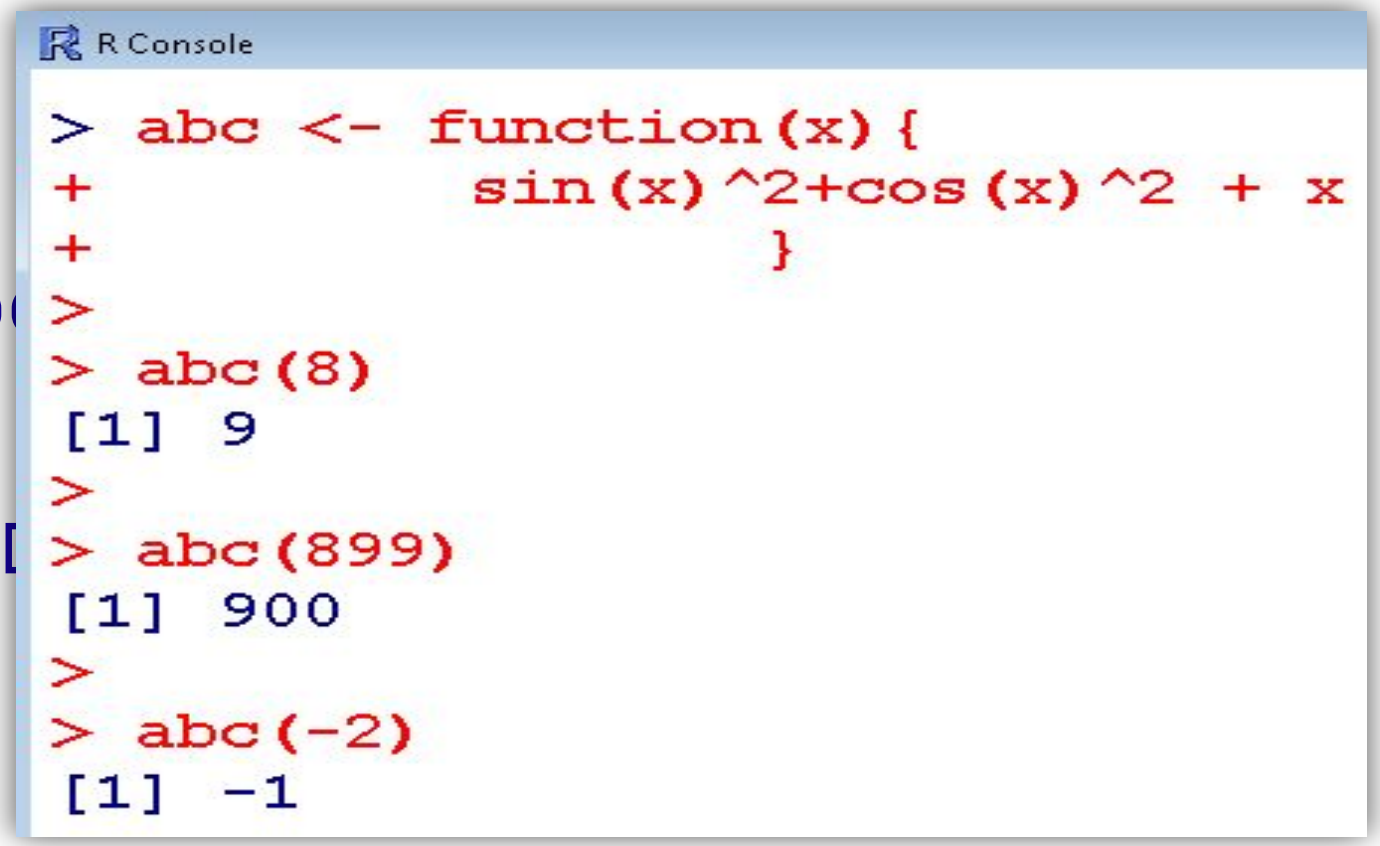
Functions- Another example

```
> abc <- function(x){
            sin(x)^2+cos(x)^2 + x
                  }

> abc(8)
  [1] 9

>abc(899)   [1] 90

> abc(-2)
```



Matrix

Matrices are important objects in any calculation.

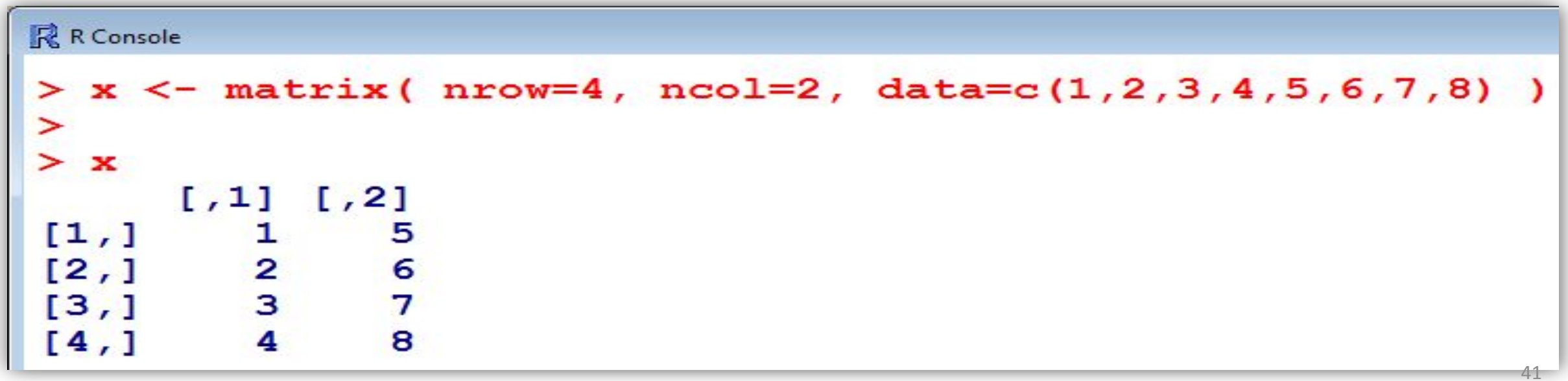
A matrix is a rectangular array with *p* rows and *n* columns.

An element in the *i*-th row and *j*-th column is denoted by X_{ij} (book version) or *X*[*i*, *j*] ("program version"), *i* = 1,2,...,*n*, *j* = 1,2,...,*p*.

An element of a matrix can also be an object, for example a string. However, in mathematics, we are mostly interested in numerical matrices, whose elements are generally real numbers

In R, a 4 × 2-matrix *X* can be created with a following command:

```
> x <- matrix( nrow=4, ncol=2,
                        data=c(1,2,3,4,5,6,7,8) )
> x
      [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
```



We see:

The parameter `nrow` defines the row number of a matrix. The parameter `ncol` defines the column number of a matrix. The parameter `data` assigns specified values to the matrix

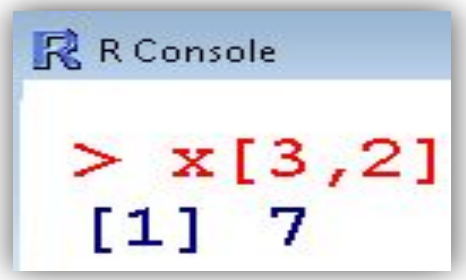
elements.

The values from the parameters are written column-wise in matrix.

```
> x
     [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
```

One can access a single element of a matrix with `x[i,j]`:

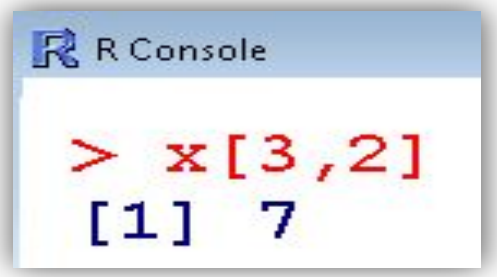
```
> x[3,2]
[1] 7
```



```
> x
     [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
```

One can access a single element of a matrix with `x[i,j]` :

```
> x[3,2]
[1] 7
```



In case, the data has to be entered row wise, then a 4 × 2-matrix *X* can be created with

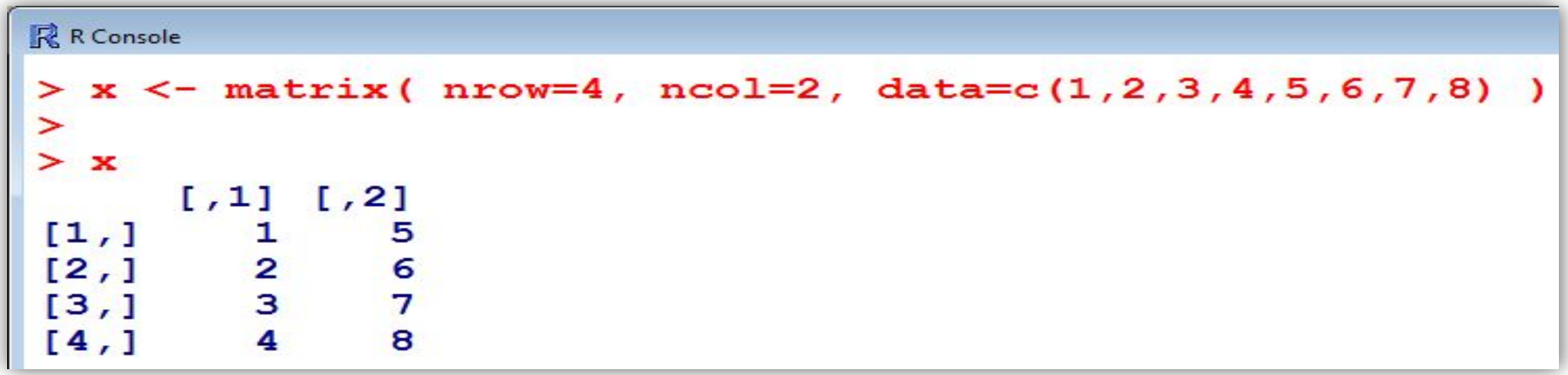
```
> x <- matrix( nrow=4, ncol=2,
             data=c(1,2,3,4,5,6,7,8),          byrow   =  TRUE)
 > x
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
```

RGui (64-bit)

```
> x <- matrix( nrow=4, ncol=2, data=c(1,2,3,4,5,6,7,8) )
> x
     [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
>
> x <- matrix( nrow=4, ncol=2, data=c(1,2,3,4,5,6,7,8), byrow = TRUE)
> x
     [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
>
```

In R, a 4 × 2-matrix *X* can be created with a following command:

```
> x <- matrix( nrow=4, ncol=2,
                        data=c(1,2,3,4,5,6,7,8) )
> x
     [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
```



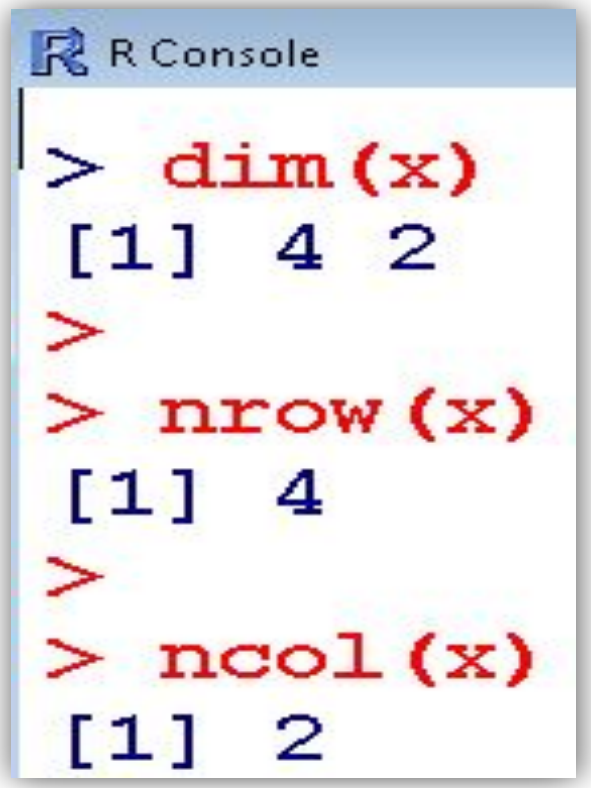
Properties of a Matrix

We can get specific *properties* of a matrix:

```
> dim(x)        # tells the
[1] 4 2           dimension of matrix

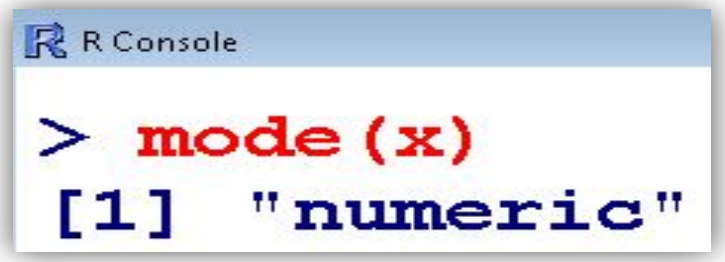

> nrow(x)       #  tells
[1] 4             the number     of   rows


> ncol(x)       #  tells
[1] 2             the number     of   columns
```



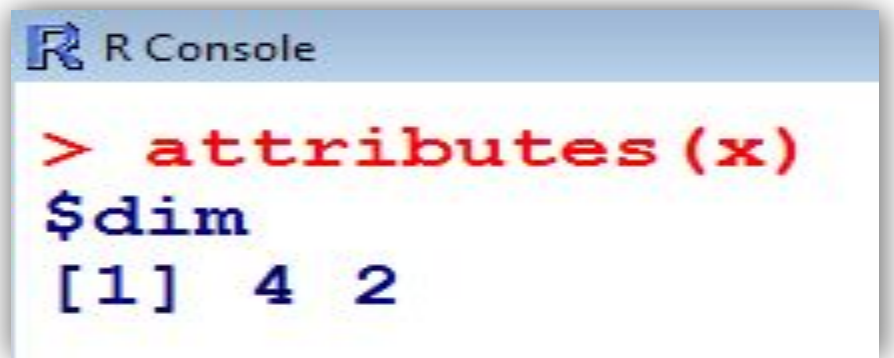
Properties of a Matrix

```
> mode(x)                 # Informs the type or storage  mode of
                            an object, e.g.,  numerical, logical
                            etc.
[1] "numeric"
```



`attributes` provides all the attributes of an object

```
> attributes(x) #Informs the dimension of matrix
$dim [1] 4 2
```



Help on the Object "Matrix"

To know more about these important objects, we use R–help on "matrix".

```
> help("matrix")
matrix  package:base  R Documentation

Matrices

Description:

'matrix' creates a matrix from the given set of  values.

'as.matrix' attempts to turn its argument into a  matrix.

'is.matrix' tests if its argument is a (strict)  matrix. It is
generic: you can write methods to  handle specific classes of
objects, see Internal  Methods.
```

Then we get an overview on how a matrix can be created and what parameters are available:

Usage:

```
matrix(data [= NA,nrow = 1,ncol = 1,byrow = FALSE,
                                    dimnames = NULL)
as.matrix(x)  is.matrix(x)
```

Arguments:

```
data: an optional data vector.  nrow: the desired number of rows
ncol: the desired number of columns
byrow: logical. If 'FALSE'(the default) the matrix  is filled by
  columns, otherwise the matrix is  filled by rows.

dimnames: A 'dimnames' attribute for the matrix: a
 'list' of length 2.



  x: an R object.
```

Then, the meaning of each parameter is explained:

Details:

If either of 'nrow' or 'ncol' is not given, an attempt is made to infer it from the length of 'data' and the other parameter.

If there are too few elements in 'data' to fill the array, then the elements in 'data' are recycled. If 'data' has length zero, 'NA' of an appropriate type is used for atomic vectors and 'NULL' for lists.

'is.matrix' returns 'TRUE' if 'x' is a matrix (i.e., it is _not_ a 'data.frame' and has a 'dim' attribute of length 2) and 'FALSE' otherwise.

'as.matrix' is a generic function. The method for data frames will convert any non-numeric/complex column into a character vector using 'format' and so return a character matrix, except that all-logical data

frames will be coerced to a logical matrix.

Finally, references and cross-references are displayed...

```
References:
  Becker, R. A., Chambers, J. M. and Wilks, A.
  R. (1988) _The New S Language_. Wadsworth &  Brooks/Cole.

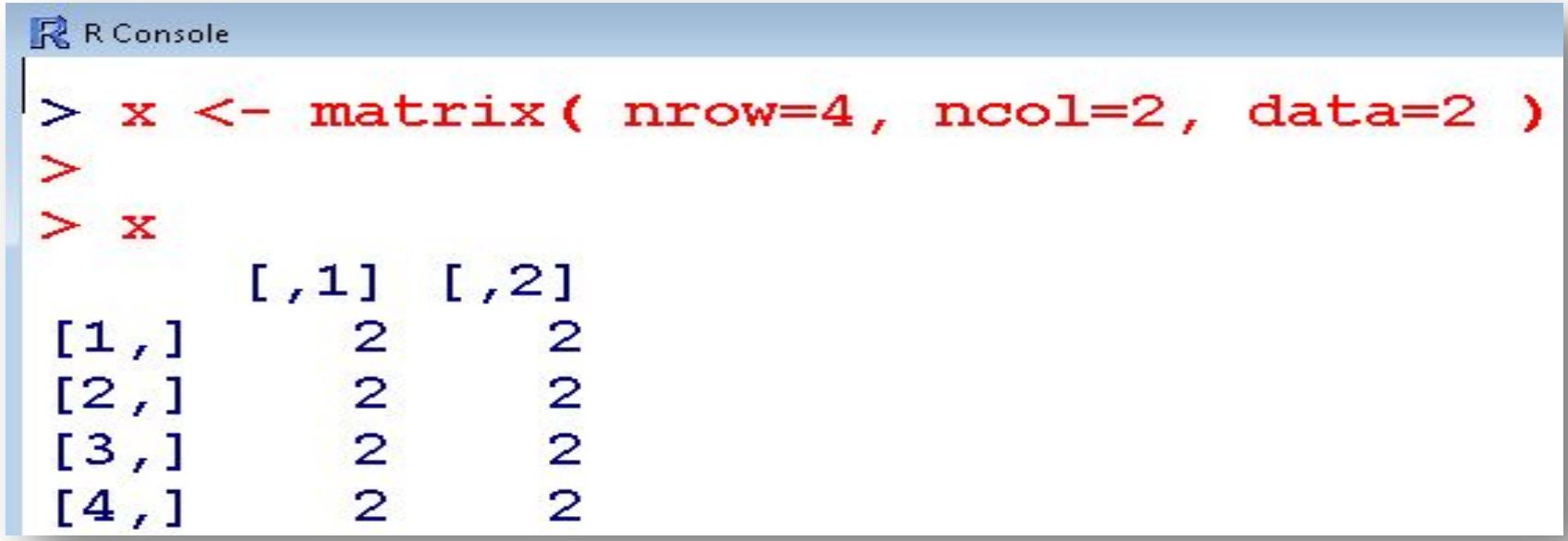
See Also:
  'data.matrix', which attempts to convert to  a numeric matrix.
```

.. as well as an example:

```
Examples:  is.matrix(as.matrix(1:10))  data(warpbreaks)
!is.matrix(warpbreaks)# data.frame, NOT matrix!
  warpbreaks[1:10,]
as.matrix(warpbreaks[1:10,]) #using
  as.matrix.data.frame(.) method
```

- **Assigning a specified number to all matrix elements:**

```
> x <- matrix( nrow=4, ncol=2, data=2 )
> x
     [,1] [,2]
[1,]    2    2
[2,]    2    2
[3,]    2    2
```

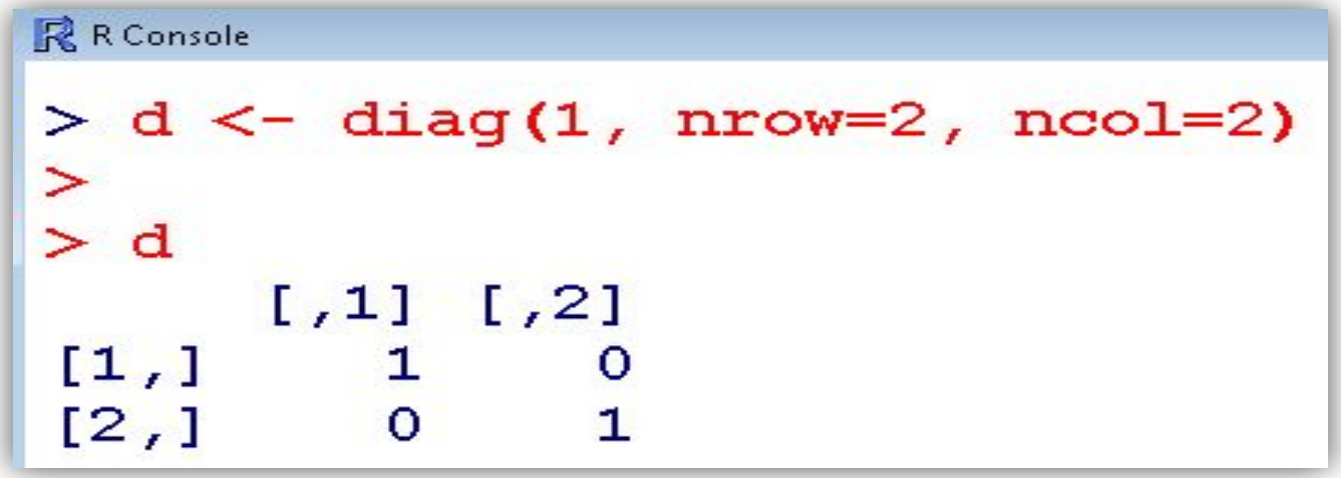


Matrix Operations

- **Construction of a diagonal matrix, here the identity matrix of a dimension 2:**

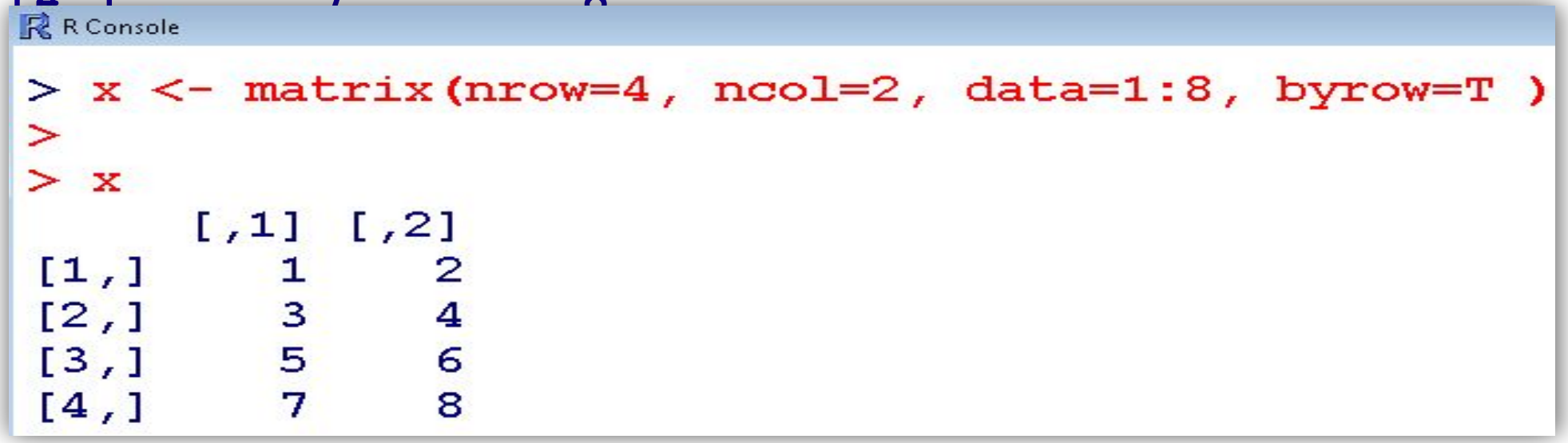
```
> d <- diag(1, nrow=2, ncol=2)
```

```
> d
     [,1] [,2]
[1,]    1    0
[2,]    0    1
```



- **Transpose of a matrix *X*: *X'***

```
> x <- matrix(nrow=4, ncol=2, data=1:8, byrow=T )
> x
     [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
```

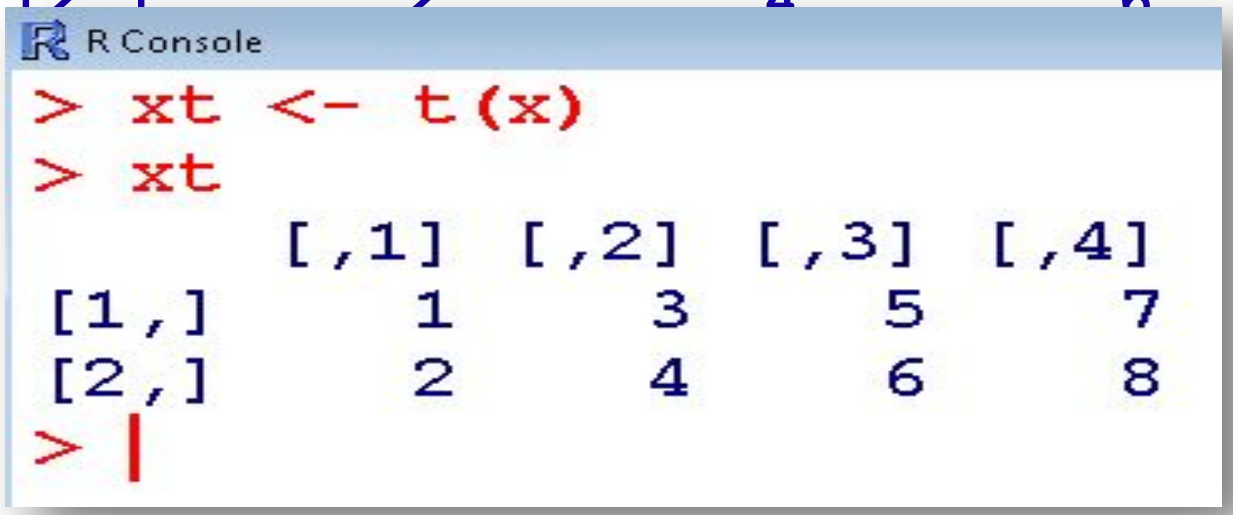


- **Transpose of a matrix *X*: *X'***

```
> xt  <-  t(x)

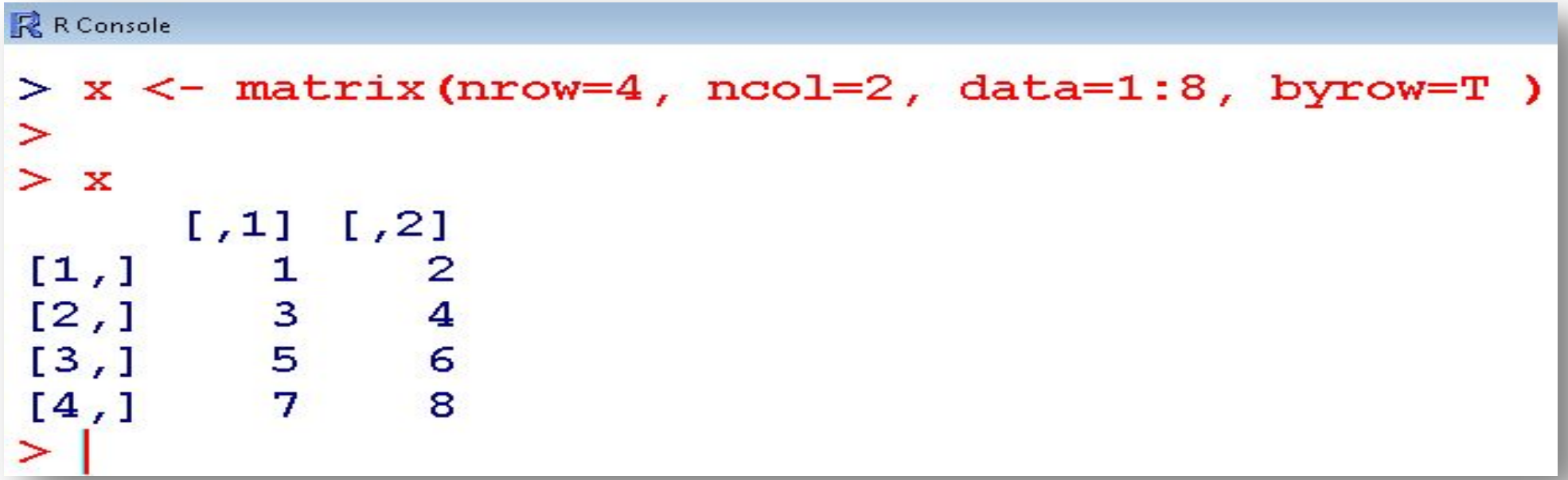
> xt

     [,1]  [,2]  [,3]  [,4]
[1,]    1     3     5     7
[2,]    2     4     6     8
```



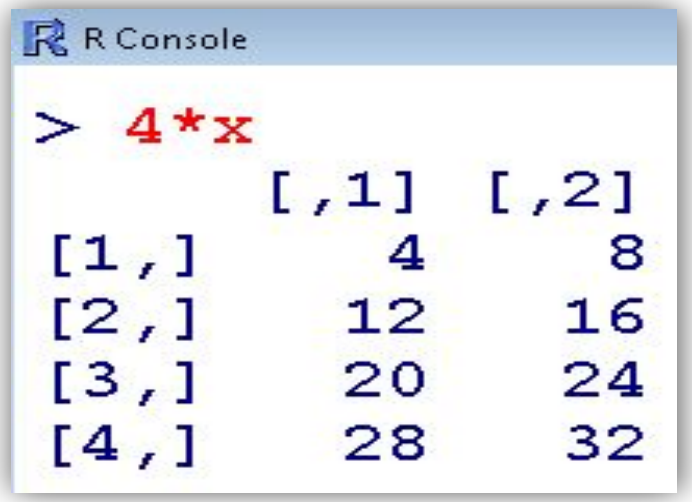
- **Multiplication of a matrix with a constant**

```
> x <- matrix(nrow=4, ncol=2, data=1:8, byrow=T )
> x
     [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
```



- **Multiplication of a matrix with a constant**

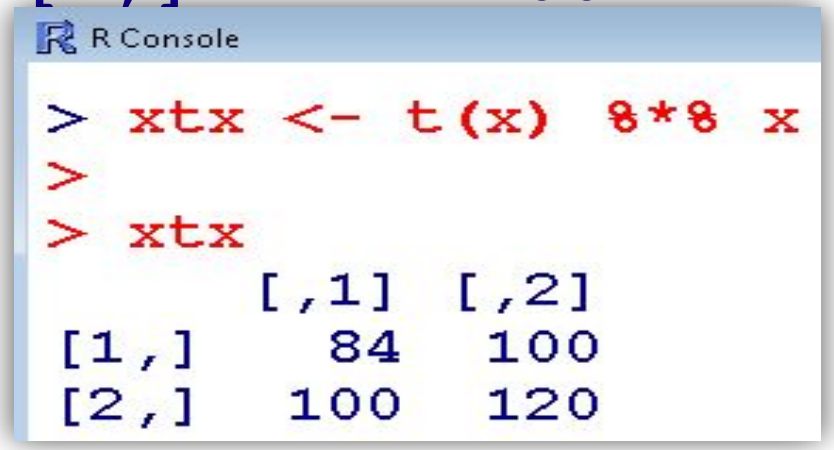
```
> 4*x
     [,1] [,2]
[1,]    4    8
[2,]   12   16
[3,]   20   24
[4,]   28   32
```



- **Matrix multiplication: operator %*%**

Consider the multiplication of *X'* with *X*

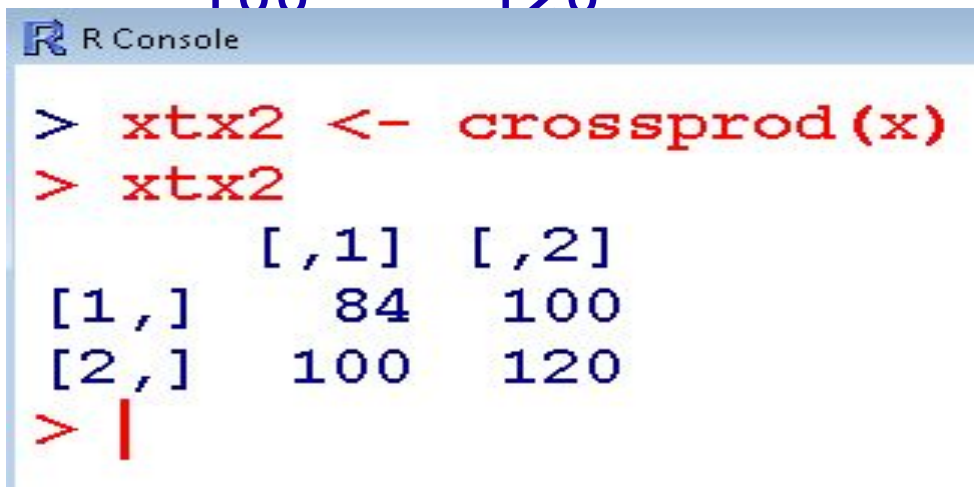
```
> xtx <- t(x)
  %*% x
> xtx
     [,1]  [,2]
[1,]   84   100
[2,]  100   120
```



- **Cross product of a matrix *X*, *X*'*X*, with a function `crossprod`**

```
> xtx2 <- crossprod(x)

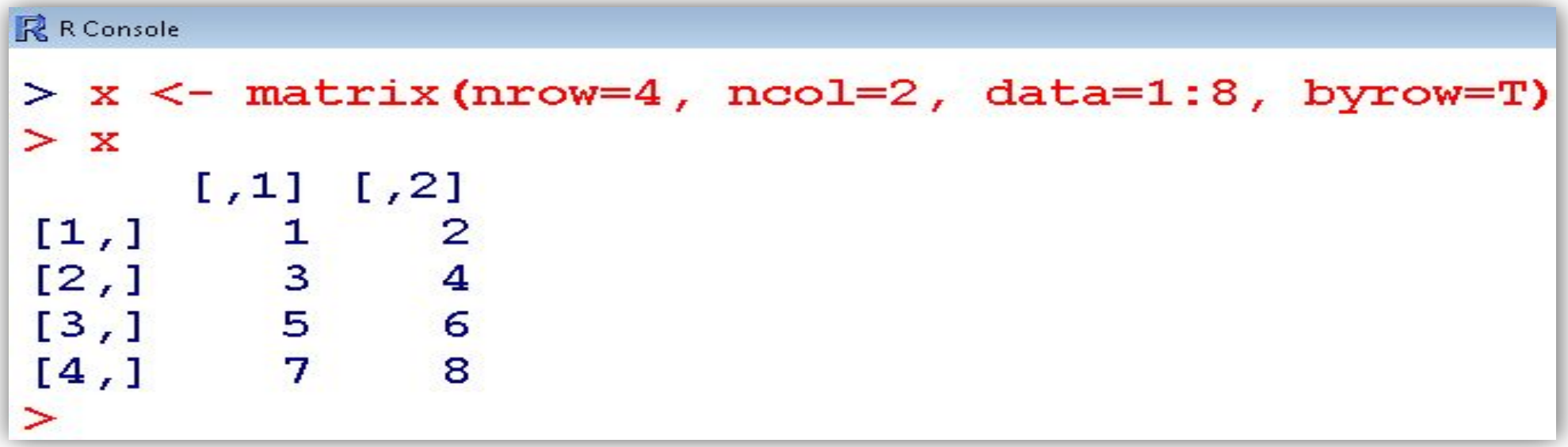
> xtx2
     [,1]  [,2]
[1,]   84   100
[2,]  100   120
```



Note: Command `crossprod()` executes the multiplication faster than the conventional method with `t(x) %*% x`

- **Addition and subtraction of matrices (of same dimensions) can be executed with the usual operators + and -**

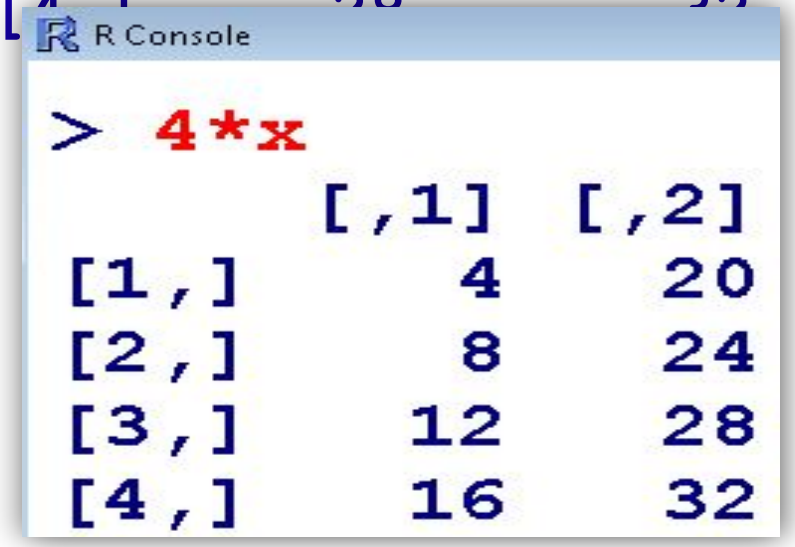
```
> x <- matrix(nrow=4, ncol=2, data=1:8, byrow=T)
> x
     [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
```



- **Addition and subtraction of matrices (of same dimensions!) can be executed with the usual operators + and -**

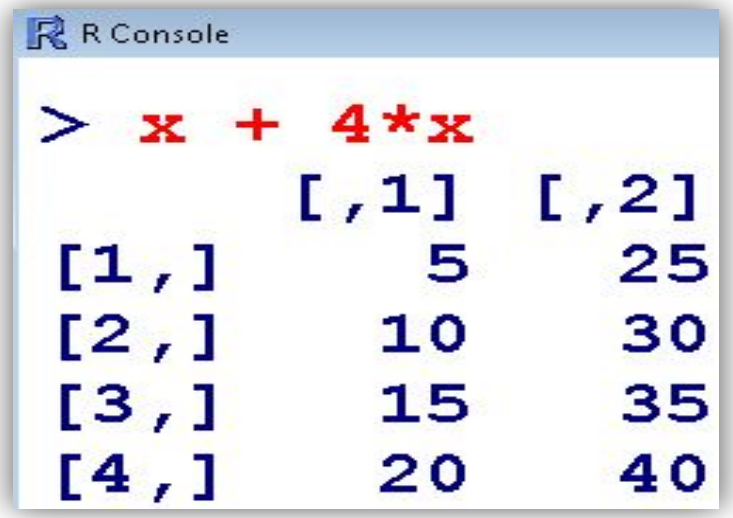
```
> 4*x
```

```
     [,1] [,2]
[1,]    4    8
[2,]   12   16
[3,]   20   24
[4,]   28   32
```

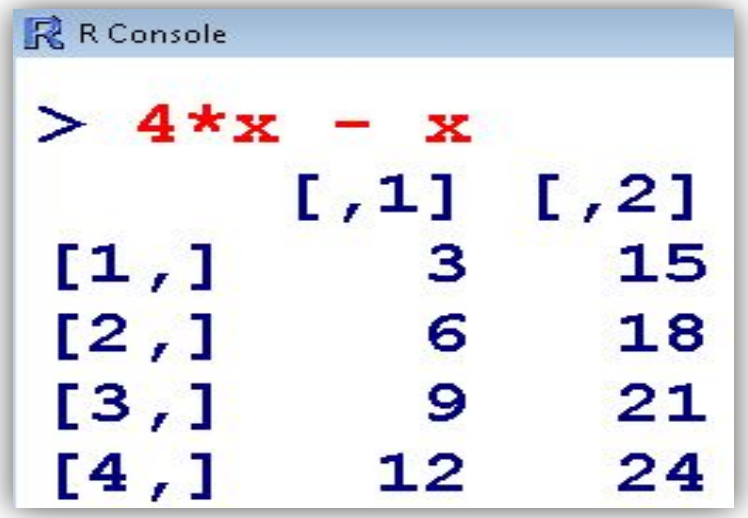


- **Addition and subtraction of matrices (of same dimensions!) can be executed with the usual operators + and -**

```
> x + 4*x
     [,1] [,2]
[1,]    5   10
[2,]   15   20
[3,]   25   30
[4,]   35   40
```

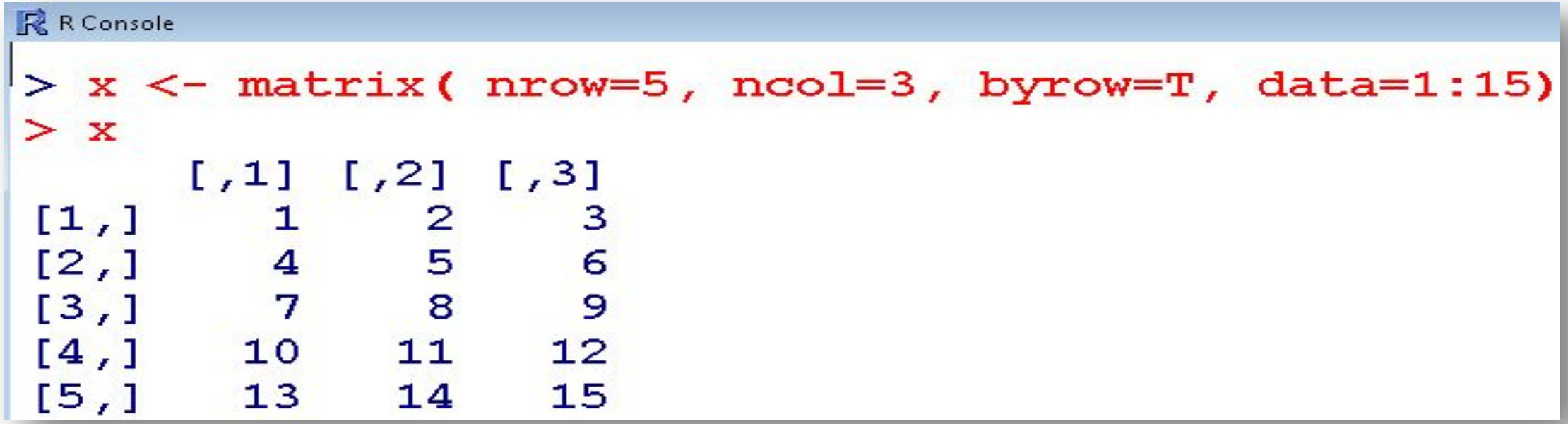



```
> 4*x - x
     [,1] [,2]
[1,]    3    6
[2,]    9   12
[3,]   15   18
[4,]   21   24
```



- **Access to rows, columns or submatrices:**

```
> x <- matrix( nrow=5, ncol=3, byrow=T, data=1:15)
> x
      [,1]  [,2]  [,3]
[1,]     1     2     3
[2,]     4     5     6
[3,]     7     8     9
[4,]    10    11    12
```



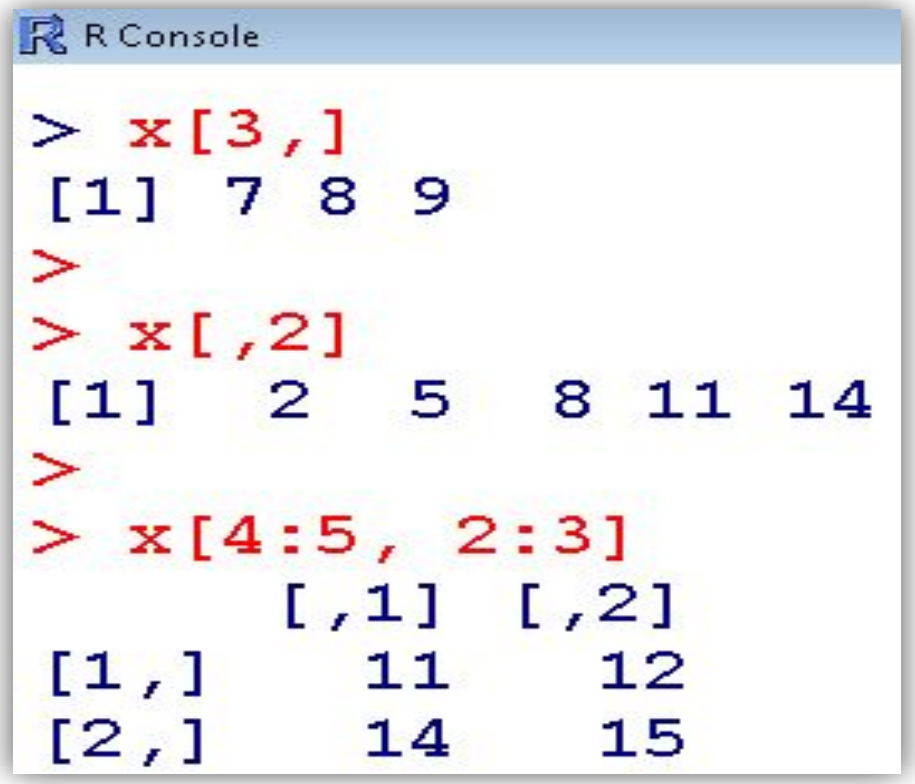
- **Access to rows, columns or submatrices:**

```
>x[3,]   [1] 7 8 9

 > x[,2]


 [1]   2  5  8  11   14

 > x[4:5,     2:3]
       [,1]     [,2]
[1,] 11          12
[2,] 14          15
```



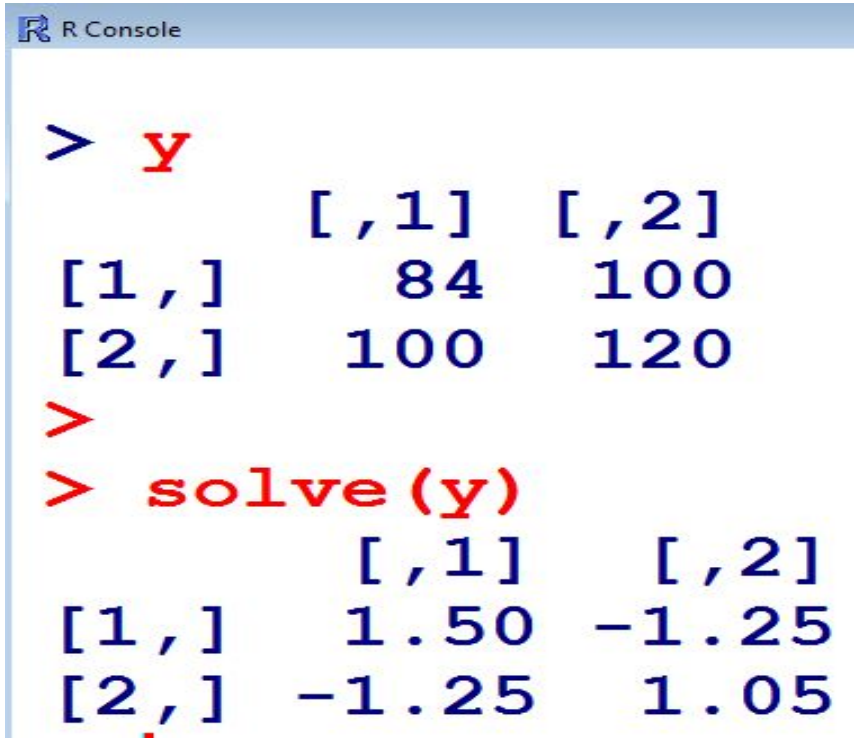
- **Inverse of a matrix:**

  `solve()` finds the inverse of a positive definite matrix **Example:**

```
>y<- matrix( nrow=2,ncol=2, byrow=T,
data=c(84,100,100,120))
```

```
> y
     [,1] [,2]
[1,]   84  100
[2,]  100  120

  > solve(y)
        [,1]    [,2]
[1,] 1.50      -1.25
```



- **Eigen Values and Eigen Vectors:**

`eigen()` **finds the eigen values and eigen vectors of a positive definite matrix**

Example:

```
> y
```

```
> eigen(y)
$values
[1] 203.6070864   0.3929136

$vectors
           [,1]       [,2]
[1,] 0.6414230 -0.7671874
[2,] 0.7671874  0.6414230
```

R Console

```
> y
     [,1] [,2]
[1,]   84  100
[2,]  100  120
> eigen(y)
$values
[1] 203.6070864   0.3929136

$vectors
           [,1]       [,2]
[1,] 0.6414230 -0.7671874
[2,] 0.7671874  0.6414230
```